

awk

- Filtro para ficheros de texto programable.
- awk selecciona campos y registros (líneas) de un fichero de entrada y el resultado se muestra en la salida estándar:
- La estructura es:

awk -parámetros *parte_dónde* *programa* nomFich

awk {acción} nomFich

awk -F {acción} nomFich

awk dir1 {acción} nomFich

awk dir1,dir2 {acción} nomFich

awk -f fichPrograma nomFich

awk

- awk considera el fichero de entrada como un conjunto de registros.
 - Por defecto, cada línea es un registro.
- awk divide los registros en campos.
 - Por defecto, cada campo es un conjunto de caracteres separado por el carácter blanco.

- Algunas variables asociadas a campos y registros:

\$0 identifica a todo el registro
\$1 identifica al campo primero
...
\$30 identifica al campo treinta

NR Número del registro
NF Número de campos del registro
FILENAME Nombre del fichero de entrada

awk

- **parte_dónde** indica las direcciones (`dir1 dir2`) donde se aplica la acción del programa.
 - Las direcciones pueden ser expresiones regulares.
 - Las direcciones pueden ser expresiones booleanas.
 - `BEGIN` indica que la acción se ejecute antes de tratar ningún registro (línea)
 - `END` indica que la acción se ejecute después de tratar todos los registros (línea)
- **acción** indica la secuencia de sentencias del programa.
 - Por defecto, cada sentencia en una línea
 - Con `;` pueden ponerse varias sentencias en una misma línea.

awk

- Sentencias del programa (**acción**) :
 - De escritura:
 - `print` con formato implícito.
 - `printf` con formato explícito (similar al lenguaje C).
 - De asignación.
 - De control de flujo.
 - `if-else`
 - `while`
 - `for ( ; ; )`
 - `for ( in )`
- Si no hay parte **{acción}**, awk ejecuta `{print}`

awk

- Funciones:
 - `length(s)` devuelve la longitud de la cadena de caracteres (*string*) `s`
 - `substr (s, p)` devuelve la parte del *string* `s` desde la posición `p` hasta el final de `s`.
 - `substr (s, p, n)` devuelve la parte del *string* `s` desde la posición `p` y de longitud `n` caracteres (como máximo)
 - `index (s, s1)` devuelve la posición del *string* `s` en que aparece el primer carácter del *string* `s1` por primera vez . Si `s1` no aparece en `s`, la función devuelve 0.
 - `n = split(s, v, sep)` divide el *string* `s` en `n` elementos `v[1]`, ... , `v[n]` usando `sep` como separador

- *string* $\equiv$ cadena de caracteres
- La primera posición de un *string* es la 1

awk

```
$cd ~/tso/01/parejas
```

```
$awk '{print substr($2,3)}' regimenenes    $awk '{print $2}' regimenenes
```

```
$awk '{print substr($2,3,4)}' regimenenes
```

```
$awk '{print "posicion=", index($1,"rne")}' regimenenes
```

```
$awk '{print "posicion=", index($2, "n")}' regimenenes
```

```
$awk '{split ($0, elem, " ")  
    print "alim=",elem[1]," --- ", "nomb=",elem[2]}' regimenenes
```

```
$awk '{n=split ($0, elem, " ")  
    print "numero de campos de este registro=",n}' regimenenes
```

awk

```
$cat fichPrueba
```

```
Esta linea tiene campos cinco
```

```
ESta linea tiene cuatro
```

```
ESTa linea tres
```

```
ESTA dos
```

```
$awk '{n=split ($0, elem, " ")
```

```
print "campos=",n, "primer=", elem[1] , "ultimo=",elem[n]}' fichPrueba
```

Redirección de entrada

- La entrada estándar se redirecciona hasta encontrar la expresión centinela:

<< centinela

```
$tr aeiou AEIOU << YA
```

```
primavera  
verano  
otoño  
Invierno  
YA
```

```
prImAvErA
```

```
vErAn0
```

```
Ot0ño0
```

```
InvIErn0
```

```
$tr aeiou AEIOU >fich << YA
```

```
primavera  
verano  
otoño  
Invierno  
YA
```

```
$cat fich
```

```
prImAvErA
```

```
vErAn0
```

```
Ot0ño0
```

```
InvIErn0
```

Comando tee

- saca dos copias de los caracteres que lee. Una copia va a la salida estándar, y la otra al fichero

tee fich

```
$cd ~/tso/01/palabras
```

```
$cat dias | tee fich
```

```
lunes  
martes  
miercoles  
jueves  
viernes  
Sabado  
domingo
```

```
$cat fich
```

```
lunes  
martes  
miercoles  
jueves  
viernes  
Sabado  
domingo
```

Comando tee

- **tee** fich saca dos copias de los caracteres que lee. Una copia va a la salida estándar (**si no está redirigida**), y la otra al fichero

```
$cd ~/tso/01/palabras
```

```
$sort dias |tee f1 | rev |tee f2 |tac |tee f3 | cut -c1-3
```

\$cat f1

domingo
jueves
lunes
martes
miercoles
sabado
viernes

\$cat f2

ognimod
seveuj
senul
setram
selocreim
odabas
senreiv

\$cat f3

senreiv
odabas
selocreim
setram
senul
seveuj
ognimod

Salida por pantalla
(después del **cut**)

sen
oda
sel
set
sen
sev
ogn

Agrupación de comandos

- Los paréntesis () permiten agrupar comandos
- El carácter ; permite separar comandos

```
$cd ~/tso/01/palabras
```

```
$pwd ; cat dias > fichDias  
/home/bx1087/tso/01/palabras
```

```
$cat fichDias
```

```
lunes  
martes  
miercoles  
jueves  
viernes  
Sabado  
domingo
```

```
$(pwd ; cat dias) > fichDias  
← No saca nada por pantalla
```

```
$cat fichDias
```

```
/home/bx1087/tso/01/palabras  
lunes  
martes  
miercoles  
jueves  
viernes  
Sabado  
domingo
```

Comando test

- **test** permite evaluar expresiones:
 - En las que se comparan números o cadena de caracteres (*strings*).
 - En las que se comprueban atributos de ficheros/directorios.
- **test** siempre devuelve un valor booleano:
 - 0 verdadero (*true*)
 - Distinto de 0 *falso (false)*.

```
$v=4
```

```
$test $v -ge 3                      # -ge      mayor o igual que  
$echo $?
```

```
0              ← verdadero(true)
```

```
$v=2
```

```
$test $v -ge 3  
$echo $?
```

```
1              ← falso(false)
```

Comando test

test expresion

o también

[expresion]

v=4

```
$[ $v -ge 3 ]
```

-ge *mayor o igual que*

```
$echo $?
```

```
0            ← verdadero(true)
```

\$v=2

```
$[ $v -ge 3 ]
```

```
$echo $?
```

```
1            ← falso(false)
```

Comando test

test

Números enteros:

x	-eq	y	x igual a y
x	-ge	y	x mayor o igual que y
x	-gt	y	x mayor que y
x	-le	y	x menor o igual que y
x	-lt	y	x menor que y
x	-ne	y	x distinto de y

Strings:

x	=	y	x igual a y
x	!=	y	x distinta de y
x			x no es <i>null</i>
-n		x	la longitud de x es mayor que 0
-z		x	la longitud de x es igual que 0

Comando test

test

ficheros:

- d** *nom* es un directorio
- f** *nom* es un fichero
- r** *nomb* tiene permiso de lectura
- w** *nomb* tiene permiso de escritura
- x** *nomb* tiene permiso de ejecución
- s** *nomb* tiene longitud distinta de 0.

operadores lógicos:

- !** *expr* *expr* no es verdadera
- e1 -a e2** e1 y e2 son verdaderas
- e1 -o e2** e1 o e2 son verdaderas

Comando test

```
$vi scriptComp
```

```
if [ $1 -eq 3 ]  
then  
    echo "$1 es igual que 3"  
else  
    echo "$1 no es igual que 3"  
fi
```

```
$bash scriptComp 4
```

```
4 no es igual que 3
```

Comando test

```
$cd ~/tso/01
```

```
$vi scriptTipoParam
```

```
if [ -d $1 ]  
then  
    echo "$1 es un directorio"  
else  
    if test -f $1  
    then  
        echo "$1 es un fichero"  
    else  
        echo "$1 no es fichero/directorio"  
    fi  
fi
```

o también **test -d \$1**

o también **[-f \$1]**

```
$bash scriptTipoParam palabras
```

```
palabras es un directorio
```

```
$bash scriptTipoParam palabras/dias
```

```
palabras/dias es un fichero
```